

Compte rendu TP 1 : SINT

David Prospérin & Angélique Parois

CENTRALE NANTES

Ce rapport a été entièrement rédigé en LaTeX.

SEC28
10/10/2026

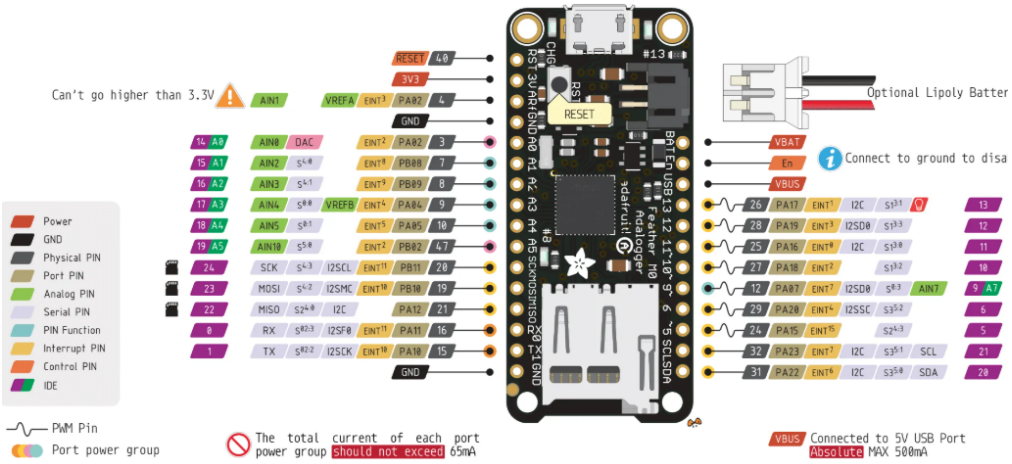
Sommaire

3	1 - Activité 1 : Retrouver un format de données
4	2 - Activité 2 : Contrôle des LEDs et affichage de l'encodeur
9	3 - Programme final
10	4 - Conclusion

1 - Activité 1 : Retrouver un format de données

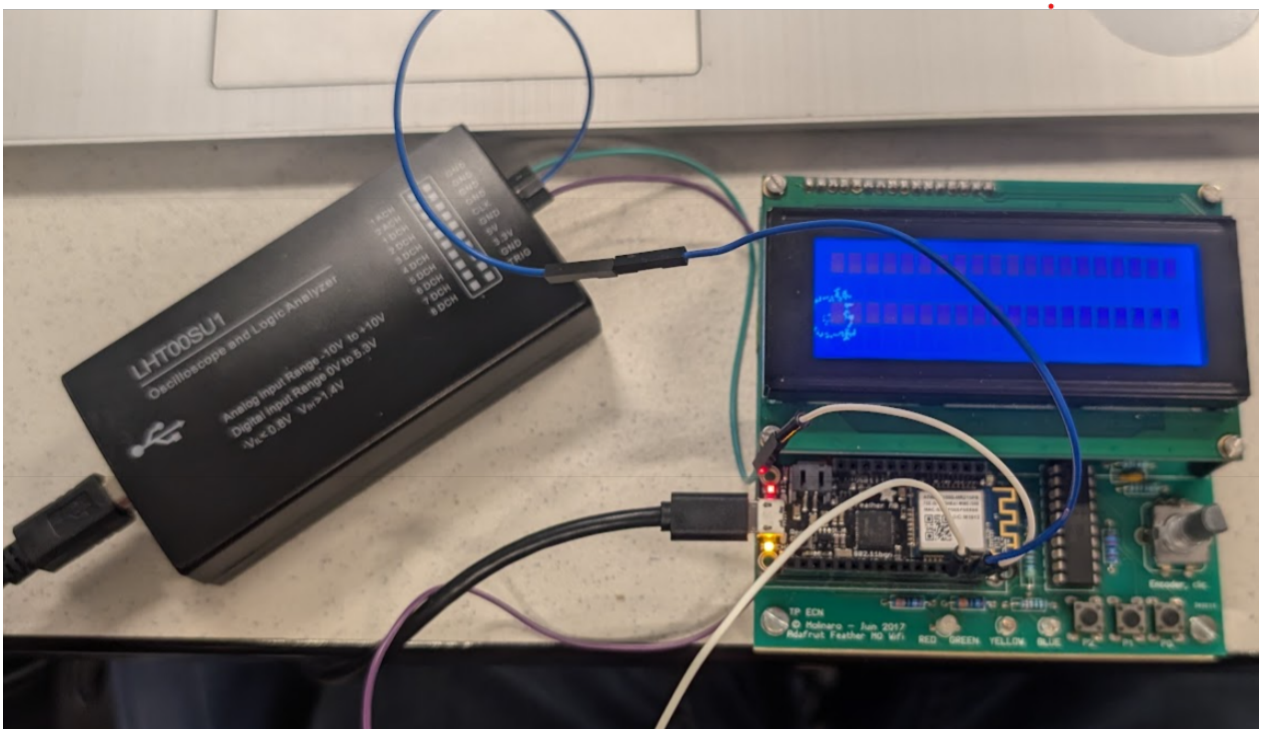
La première mission était de retrouver la configuration de la liaison série et le format des données transmises par le PIC. Pour ce TP nous avons utilisé la carte Feather M0 et un analyseur logique.

Tout d'abord nous avons branché l'analyseur logique aux broches de la carte Feather M0 qui sont connectées au PIC. Ces broches correspondent à la liaison série `Serial1`.

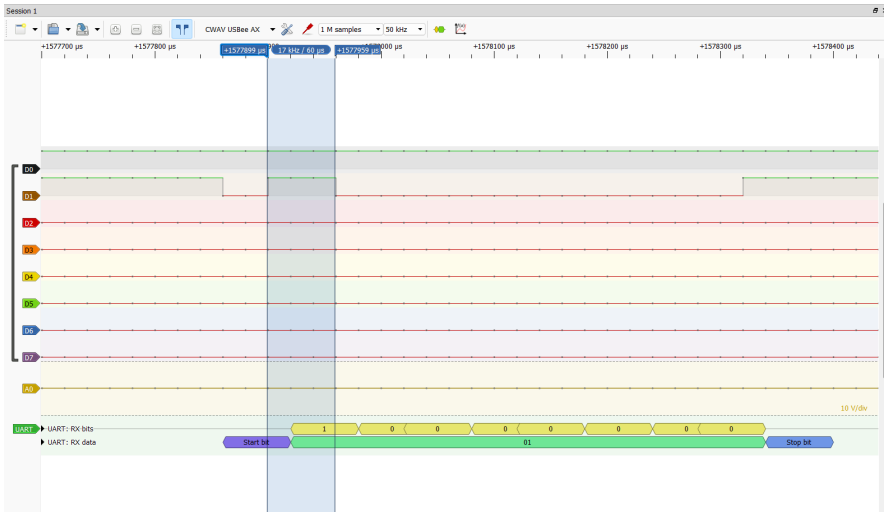


Le pinout de la carte nous montre que la broche 16 est associée à RX et que la broche 15 est associée à TX.

Comme le montre la photo suivante, on connecte deux broches d'acquisition numérique D0 et D1 aux broches 15 et 16 de la Feather M0.



Première étape *— on détermine la vitesse de transmission des trames à l'aide de PulseView. Pour cela on lance une acquisition durant laquelle on appuie sur un bouton. On obtient une trame avec 1 bit à 1, puis on mesure le temps à l'état haut.*



On obtient une période de 60×10^{-6} s. Avec un produit en croix on calcule la vitesse de transmission :

$$\frac{1}{60 \times 10^{-6}} \approx 16\,600 \text{ bauds} \tag{1}$$

On s'appuie sur la table suivante pour déterminer la vitesse standard de la liaison UART. On arrondit à 19 200 bauds, qui est une vitesse de transmission standard.

Transmission speed				Real transmission speed	
Bauds	**Bit/s**	**Bit duration**	**Speed**	**Speed**	**Byte duration**
50 Bd	50 bits/s	20.000 ms	6.25 bytes/s	5 bytes/s	200.000 ms
75 Bd	75 bits/s	13.333 ms	9.375 bytes/s	7.5 bytes/s	133.333 ms
110 Bd	110 bits/s	9.091 ms	13.75 bytes/s	11 bytes/s	90.909 ms
134 Bd	134 bits/s	7.463 ms	16.75 bytes/s	13.4 bytes/s	74.627 ms
150 Bd	150 bits/s	6.667 ms	18.75 bytes/s	15 bytes/s	66.667 ms
200 Bd	200 bits/s	5.000 ms	25 bytes/s	20 bytes/s	50.000 ms
300 Bd	300 bits/s	3.333 ms	37.5 bytes/s	30 bytes/s	33.333 ms
600 Bd	600 bits/s	1.667 ms	75 bytes/s	60 bytes/s	16.667 ms
1200 Bd	1200 bits/s	833.333 µs	150 bytes/s	120 bytes/s	8.333 ms
1800 Bd	1800 bits/s	555.556 µs	225 bytes/s	180 bytes/s	5.556 ms
2400 Bd	2400 bits/s	416.667 µs	300 bytes/s	240 bytes/s	4.167 ms
4800 Bd	4800 bits/s	208.333 µs	600 bytes/s	480 bytes/s	2.083 ms
9600 Bd	**9600 bits/s**	**104.167 µs**	**1200 bytes/s**	**960 bytes/s**	**1.042 ms**
19200 Bd	19200 bits/s	52.083 µs	2400 bytes/s	1920 bytes/s	520.833 µs
28800 Bd	28800 bits/s	34.722 µs	3600 bytes/s	2880 bytes/s	347.222 µs
38400 Bd	38400 bits/s	26.042 µs	4800 bytes/s	3840 bytes/s	260.417 µs
57600 Bd	57600 bits/s	17.361 µs	7200 bytes/s	5760 bytes/s	173.611 µs
76800 Bd	76800 bits/s	13.021 µs	9600 bytes/s	7680 bytes/s	130.208 µs
115200 Bd	**115200 bits/s**	**8.681 µs**	**14400 bytes/s**	**11520 bytes/s**	**86.806 µs**
230400 Bd	230400 bits/s	4.340 µs	28800 bytes/s	23040 bytes/s	43.403 µs
460800 Bd	460800 bits/s	2.170 µs	57600 bytes/s	46080 bytes/s	21.701 µs
576000 Bd	576000 bits/s	1.736 µs	72000 bytes/s	57600 bytes/s	17.361 µs
921600 Bd	921600 bits/s	1.085 µs	115200 bytes/s	92160 bytes/s	10.851 µs

Deuxième étape — *on affiche la trame UART avec la vitesse de transmission déterminée précédemment. Quand on appuie sur les différents boutons, on reçoit deux trames : la première donne l'information sur le bouton poussé, la deuxième notifie que le bouton est relâché.*

Lorsqu'on appuie sur deux boutons en même temps, un bouton est forcément poussé quelques millisecondes avant l'autre : celui-ci envoie sa trame, puis on reçoit la trame du bouton 2. Une fois qu'on a lâché l'un des deux boutons, par exemple le 2, on reçoit la trame du bouton 1 qui est toujours pressé. Lorsque ce dernier est lui aussi relâché, on reçoit une trame nulle pour indiquer que tous les boutons ont été relâchés.

Exemple — *en appuyant sur P1 P0, on reçoit 2 1, 1 0.*

On sait déjà que la taille de la trame est d'un octet. Après nos tests, on a déterminé que la trame possède un bit de start et un bit de stop. On en déduit que la longueur totale de la trame est de 10 bits.

2 - Activité 2 : Contrôle des LEDs et affichage de l'encodeur

En utilisant le projet de départ, on crée un fichier `io.cpp` pour implémenter les fonctions déclarées dans `io.h` :

> *`beginIO()` configure la liaison série 1 (entre le PIC et le Feather) ;*
> *`ledWrite()` allume ou éteint individuellement chaque LED ;*
> *`pushButton()` permet de savoir si un poussoir est appuyé ou non ;*

`encoderValue()` retourne la valeur cumulée de l'encodeur, limitée entre 0 et 63.

Fonction beginIO

Comme montré dans la première activité, la vitesse de transmission entre le micro et le PIC est de 19 200 bauds. De plus, il n'y a pas de bit de parité, un bit de start et un bit de stop. On choisit alors la configuration suivante pour la liaison série :

```
void beginIO (void) {
    Serial1.begin(19200, SERIAL_8N1);
}
```

Fonction ledWrite

Pour rédiger cette fonction, nous nous sommes aidés de la description de la trame à envoyer ci-dessous pour allumer les LEDs.

Numéro du bit	7	6	5	4	3	2	1	0
Valeur	0	0	0	0	0	0 : éteindre 1 : allumer	00 : led RED 01 : led GREEN 10 : led YELLOW 11 : led BLUE	

TAB. 1: Description de l'octet de contrôle des leds

Le paramètre `inLed` correspond à l'identifiant de la LED à allumer ou éteindre. Le paramètre `inLevel` correspond au niveau logique à appliquer à la LED sélectionnée. Cette fonction réalise un décalage de 2 car l'état allumé ou éteint est au bit 2 et l'identifiant de la LED est situé aux bits 1 et 0.

```
void ledWrite (const byte inLed, const bool inLevel) {
  const int byteInLevel = inLevel << 2;
  const int byte = byteInLevel | inLed;

  Serial1.write(byte);
}
```

Test de la fonction ledWrite

Le code de test de cette fonction éteint toutes les LEDs à l'allumage de la carte. Ensuite, dans la boucle principale, on a testé l'allumage de toutes les LEDs une par une pour attester de leur bon fonctionnement.

```
void setup() {
  pinMode (LED_BUILTIN, OUTPUT);
  digitalWrite (LED_BUILTIN, HIGH) ;
  Serial.begin (9600) ;
  beginIO();
  ledWrite(0, 0);
  ledWrite(1, 0);
  ledWrite(2, 0);
  ledWrite(3, 0);
}

void loop() {
  ledWrite(X, 1);
  delay(100);
}
```

La variable `X` correspond à l'identifiant de la LED allumée.



X = 0



X = 1



X = 2



X = 3

Fonction pushButton

Cette fonction permet de savoir si un poussoir est appuyé ou non. Notre principale contrainte est qu'on ne peut pas utiliser d'interruption sur front pour savoir quel bouton est pressé ou relâché. Dans notre cas, c'est le PIC qui nous renvoie via UART quel poussoir ou le CLIC de l'encodeur a été pressé ou relâché. Il faut donc continuellement recevoir et stocker les octets reçus.

On utilise la méthode `Serial1.available()`, qui retourne le nombre d'octets présents dans le buffer et s'incrémente lorsqu'un octet est reçu. La méthode `Serial1.read()`, quant à elle, dépile le buffer. Dès qu'on reçoit un octet via `Serial1`, il est placé dans la variable globale `etatButton`.

```
void loop() {
  if (Serial1.available() > 0) {
    etatButton = Serial1.read();

    Serial.print("etat recu : ");
    Serial.println(etatButton);
  }
}
```

Cette fonction réalise un masque pour chaque bouton et le CLIC :

Octet reçu	Bouton / CLIC
0x8	CLIC
0x4	P2
0x2	P1
0x1	P0

On réalise des masques pour chaque bouton dans le but de pouvoir détecter les appuis simultanés sur plusieurs boutons.

```
bool pushButton (const byte inButton) {
    return ((etatButton & 0x1) == inButton) ||
        ((etatButton & 0x2) == inButton) ||
        ((etatButton & 0x8) == inButton);
}
```

Test de la fonction pushButton

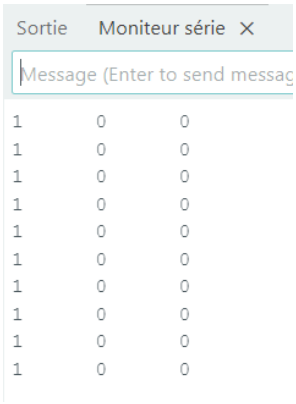
Le protocole de test consiste à afficher dans la liaison série PC ↔ Feather M0 l'état des trois boutons à l'aide du code ci-dessous.

```
void loop() {
  if (Serial1.available() > 0) {
    etatButton = Serial1.read();
    Serial.print("etat recu : ");
    Serial.println(etatButton);
```

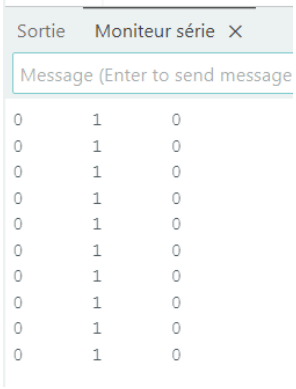
```
  }

  Serial.print(pushButton(1));
  Serial.print(" ");
  Serial.print(pushButton(2));
  Serial.print(" ");
  Serial.println(pushButton(4));
}
```

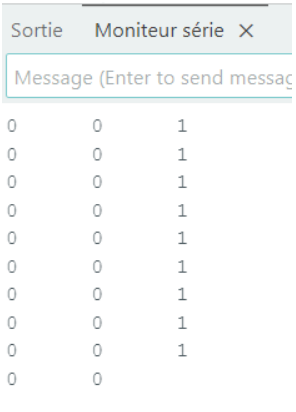
Pour valider la fonction, on a successivement pressé puis relâché les trois boutons dans l'ordre. Puis plusieurs boutons ont été pressés simultanément, comme le montrent les captures d'écran suivantes de la liaison série.



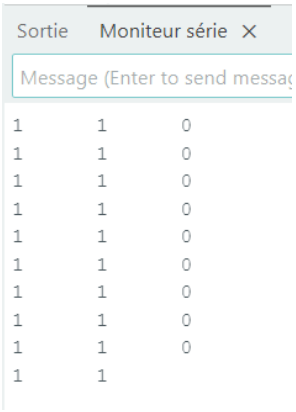
P0 pressé



P1 pressé



P2 pressé



P0 et P1 pressés

Fonction encoderValue

Cette dernière fonction retourne la valeur cumulée de l'encodeur, limitée entre 0 et 63. On utilise le même mécanisme que pour la gestion des boutons : la variable `etatButton` sert également à obtenir le sens de rotation (CW ou CCW). Lorsque le codeur tourne dans le sens horaire, le PIC envoie la valeur 0x10 et, dans le sens opposé, la valeur 0x20.

Le choix adopté consiste à ajouter 1 lors d'un cran dans le sens horaire et à retirer 1 dans l'autre sens. Une seconde contrainte est de borner la valeur `encoder` entre 0 et 63.

```
if (Serial1.available() > 0) {
    etatButton = Serial1.read();

    if (etatButton == 0x10) {
      encoder++;
    } else if (etatButton == 0x20) {
      encoder--;
    }

    if (encoder < 0) {
      encoder = 63;
    } else if (encoder > 63) {
      encoder = 0;
    }
  }
```

La fonction `encoderValue` se contente donc de retourner la valeur de la variable globale `encoder`.

```
byte encoderValue (void) {
  return encoder;
}
```

Test de la fonction encoderValue

Le test de cette fonction consiste à envoyer dans la console série la valeur courante de la variable `encoder`. Cette variable est actualisée à chaque rotation d'un cran.

```
void loop() {
  if (Serial1.available() > 0) {
    etatButton = Serial1.read();

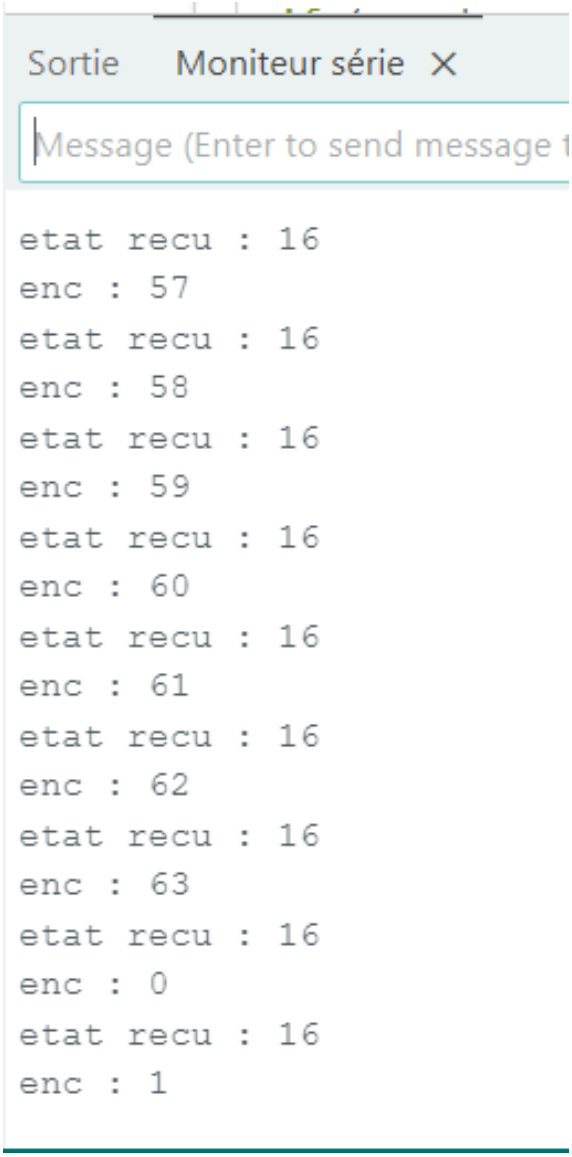
    if (etatButton == 0x10) {
      encoder++;
    } else if (etatButton == 0x20) {
      encoder--;
    }

    if (encoder < 0) {
      encoder = 63;
    } else if (encoder > 63) {
      encoder = 0;
    }

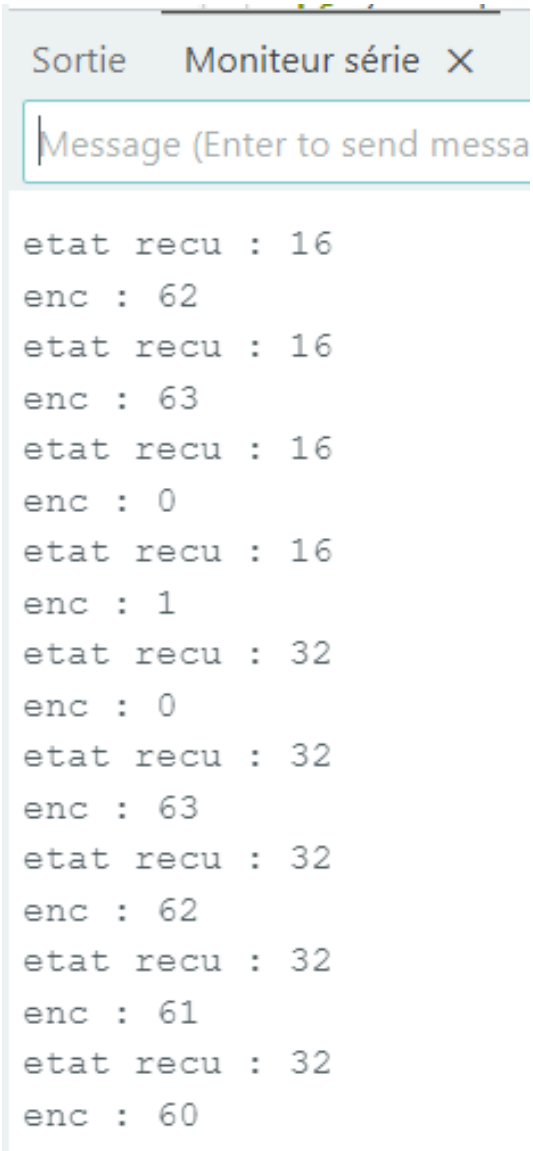
    Serial.print("etat recu : ");
    Serial.println(etatButton);
    Serial.print("enc : ");
    Serial.println(encoder);
  }

}
```

Les deux captures d'écran ci-dessous montrent bien le respect du cahier des charges.



Rotation dans le sens horaire



Rotation dans le sens anti-horaire

3 - Programme final

Le code final reprend les fonctions décrites dans les activités précédentes. Pour le clignotement de la LED BUILTIN à 1 Hz, la stratégie adoptée est de tester toutes les 500 ms s'il faut permuter la LED. On ne peut pas utiliser d'attente bloquante, ce qui bloquerait l'acquisition de l'état des boutons et du codeur.

```
void loop() {
  if (Serial1.available() > 0) {
    etatButton = Serial1.read();

    if (etatButton == 0x10) {
      encoder++;
    } else if (etatButton == 0x20) {
      encoder--;
    }

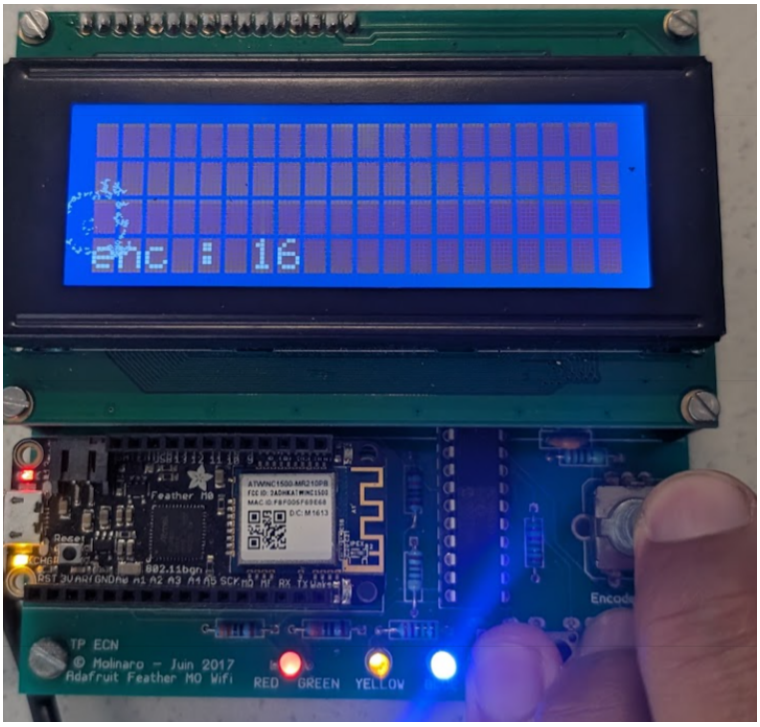
    if (encoder < 0) {
      encoder = 63;
    } else if (encoder > 63) {
      encoder = 0;
    }

    lcd.setCursor(0, 3);
    lcd.printf("enc : %d", encoderValue());
  }

    // Builtin led 1 hz
     if (millis() - led_1hz >= 500)
    {
      digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
      led_1hz = millis();
    }

    if (pushButton(8)) {
      ledWrite(3, 1);
    } else if (pushButton(1)) {
      ledWrite(2, 1);
    } else if (pushButton(2)) {
      ledWrite(1, 1);
    } else if (pushButton(4)) {
      ledWrite(0, 1);
    } else {
      ledWrite(0, 0);
      ledWrite(1, 0);
      ledWrite(2, 0);
      ledWrite(3, 0);
    }
}
```

La photo ci-dessous montre bien que le code final remplit le cahier des charges. L'appui sur le CLIC provoque l'allumage de la LED bleue. On voit la valeur courante du codeur sur la dernière ligne de l'écran LCD. La LED BUILTIN clignote bien à 1 Hz, et les LEDs jaune et rouge s'allument lors de l'appui respectif des boutons.



4 - Conclusion

Ce TP nous a permis de retrouver le format des données échangées entre le PIC et la Feather M0. À l'aide de l'analyseur logique et de PulseView, nous avons mesuré une période de 60 µs, soit une vitesse d'environ 16 600 bauds, arrondie à la vitesse standard de 19 200 bauds. L'analyse des trames a confirmé une configuration 8N1 : un bit de start, huit bits de données et un bit de stop, soit 10 bits par trame.

À partir de ces observations, nous avons implémenté les fonctions de `io.cpp` : `beginIO` pour configurer la liaison série, `ledWrite` pour commander les LEDs, `pushButton` pour lire l'état des poussoirs et du CLIC, et `encoderValue` pour suivre la valeur de l'encodeur, bornée entre 0 et 63. Chaque fonction a été validée par un test dédié via la liaison série.

Enfin, le programme final regroupe ces fonctions sans attente bloquante : le clignotement de la LED BUILTIN à 1 Hz repose sur `millis()`, ce qui permet de continuer à lire les boutons et l'encodeur en continu. Le comportement observé (LEDs, valeur de l'encodeur sur l'écran LCD) respecte le cahier des charges.